S3Documento de Diseño Técnico del Proyecto

Pipeline ETL + Chunking + Embeddings + Indexación para repositorios documentales orientados a RAG

Curso: Diseño de Infraestructura Escalable BSG Institute
Proyecto 12 – LLM Data Engineering Pipeline
Autor: Emerson Olortegui
Fecha: 10/06/2026

Índice de secciones

1. Caso de uso
2. Arquitectura
3. Diseño ETL
4. Patrones LLM
5. Resultados de chunking
6. Comparación embeddings
7. Benchmarking de indexación
8. Versionamiento
9. Lecciones aprendidas

1. Caso de uso

El caso de uso seleccionado consiste en implementar un pipeline de ingeniería de datos para LLMs que permita consultar, mediante lenguaje natural, documentos funcionales de robots RPA. Para este proyecto se trabaja con documentación en formato PDF, principalmente PDDs, los cuales describen el objetivo del robot, reglas de negocio, entradas, salidas, excepciones, validaciones y comportamiento esperado del proceso automatizado.

El problema que se busca resolver es que la documentación funcional suele estar distribuida en archivos extensos. Cuando un usuario funcional, desarrollador, auditor o evaluador necesita conocer rápidamente qué hace un robot, qué sistemas utiliza o qué excepciones maneja, normalmente debe revisar manualmente el documento completo. La solución propuesta transforma esos PDFs en texto limpio, chunks semánticos, embeddings e índice vectorial, de manera que el contenido pueda ser consultado posteriormente mediante un servicio RAG.

Elemento	Descripción
Fuente documental	PDFs funcionales de robots RPA cargados en Amazon S3 bajo el prefijo raw/.
Usuarios objetivo	Administrador técnico, analista funcional, desarrollador RPA, auditor o profesor evaluador.
Pregunta de negocio	¿Qué información relevante del documento puede recuperarse automáticamente sin leer todo el PDF?
Resultado esperado	Respuesta en lenguaje natural sustentada en chunks recuperados desde ChromaDB.
Alcance académico	Validación funcional con un conjunto pequeño de PDFs y arquitectura preparada para escalar a un repositorio mayor de documentos funcionales RPA.

El alcance se concentró en PDFs porque representan el formato principal de los documentos funcionales utilizados en la práctica del proyecto. Aunque el enunciado del curso menciona repositorios amplios y formatos como Word o HTML, para mantener un alcance controlado, demostrable y alineado a la realidad del caso de uso se priorizó el procesamiento de PDFs. Esta decisión reduce variabilidad en parsing y permite concentrar el esfuerzo en chunking semántico, embeddings, indexación y consulta RAG.

1.1 KPIs del caso de uso

- Tiempo total de procesamiento por lote de documentos.
- Cantidad de documentos procesados correctamente frente a documentos con error.
- Cantidad total de chunks generados por documento y distribución de tamaños.
- Cobertura de metadatos mínimos: nombre del documento, sección, chunk_id, fecha de procesamiento y versión de índice.
- Cantidad de chunks vacíos o descartados durante el procesamiento.
- Latencia de búsqueda semántica durante una consulta RAG.
- Costo estimado de ejecución considerando infraestructura y modelo seleccionado.

Estos indicadores permiten evaluar la solución no solo como una demostración técnica, sino como un pipeline auditable. En una implementación real, los KPIs permitirían comparar ejecuciones, detectar degradación del parsing, controlar el crecimiento del índice y estimar el costo por lote de documentos.

2. Arquitectura

La arquitectura se diseñó sobre AWS con una separación clara entre almacenamiento documental, procesamiento batch, repositorio de imágenes, observabilidad, almacenamiento vectorial y servicio de consulta RAG. La implementación base puede ejecutarse mediante tareas ECS Fargate en secuencia. Como mejora futura, se propone incorporar Amazon EventBridge y AWS Step Functions para automatizar la ejecución batch y controlar dependencias entre etapas.

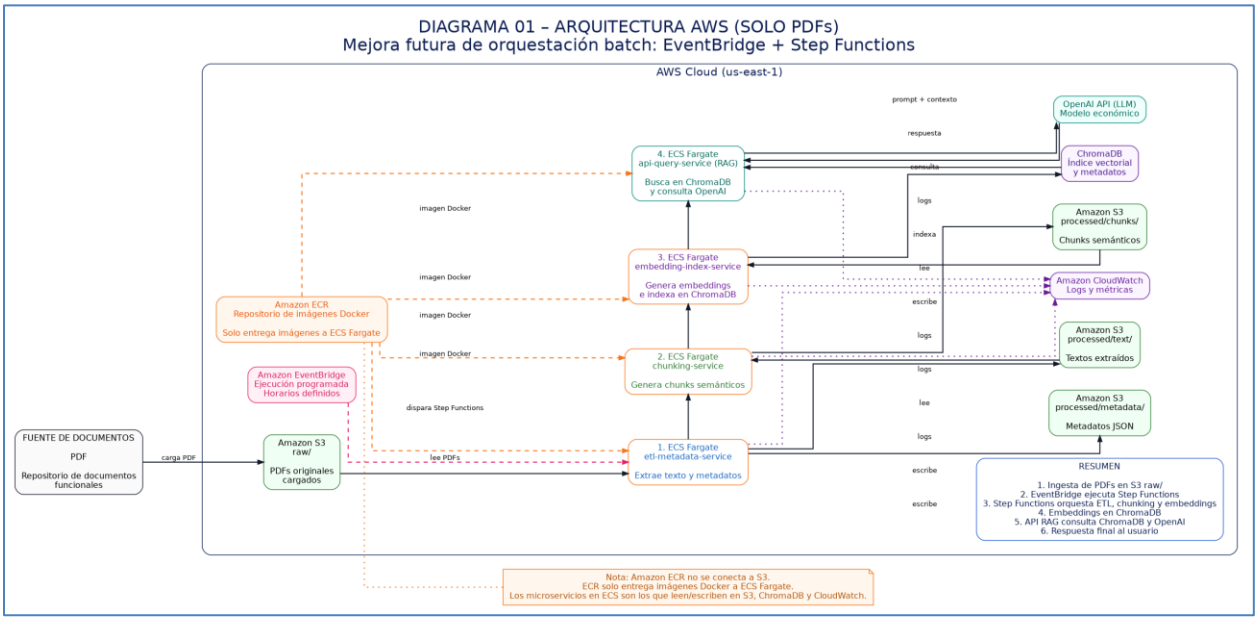


Figura 1. Arquitectura AWS actualizada: ECR entrega imágenes a ECS Fargate; los microservicios leen/escriben artefactos en S3; embedding-index-service publica ChromaDB en processed/chroma_db/ y rag-api-service lo restaura desde S3 antes de responder consultas.

La figura debe interpretarse separando tres flujos. El flujo de datos documental inicia en S3 raw/, pasa por ETL, chunking e indexación, y deja artefactos en processed/. El flujo de persistencia vectorial queda resuelto mediante S3: embedding-index-service genera la carpeta local temp/chroma_db y la sube a processed/chroma_db/; rag-api-service descarga ese prefijo al iniciar hacia /app/temp/chroma_db o temp/chroma_db, según la ruta configurada en la task definition. El flujo de despliegue parte desde Amazon ECR hacia ECS Fargate, ya que ECR almacena las imágenes Docker que ECS ejecuta; ECR no almacena documentos ni bases vectoriales.

2.1 Componentes de infraestructura AWS

Componente AWS	Responsabilidad en la solución
Amazon S3 raw/	Almacena los PDFs originales cargados para procesamiento.
Amazon S3 processed/text/	Almacena el texto plano extraído desde los PDFs.
Amazon S3 processed/metadata/	Almacena metadatos generados durante el procesamiento.
Amazon S3 processed/chunks/	Almacena archivos JSON con chunks semánticos generados por el chunking-service.
Amazon s3/**chroma_db/**	Almacena la base de datos vectorial
Amazon ECS Fargate	Ejecuta los microservicios contenerizados sin administrar servidores.
Amazon ECR	Repositorio de imágenes Docker consumidas por ECS Fargate. No se conecta funcionalmente con S3.
Amazon CloudWatch	Centraliza logs y métricas de ejecución de los servicios.
Amazon EventBridge	Mejora futura para programar ejecuciones batch por horario.
AWS Step Functions	Mejora futura para orquestar ETL, chunking e indexación con reintentos y trazabilidad.

2.2 Microservicios desarrollados en Python

Además de los servicios administrados de AWS, la solución incluye microservicios desarrollados en Python y contenerizados con Docker. Estos componentes representan la lógica propia del proyecto y son ejecutados sobre ECS Fargate.

Microservicio Python	Responsabilidad functional
etl-metadata-service	Lee PDFs desde S3 raw/, extrae texto con PyMuPDF, genera metadatos y publica

Microservicio Python	Responsabilidad functional
	salidas en processed/text/ y processed/metadata/.
chunking-service	Lee texto procesado, aplica chunking semántico y publica archivos *_chunks.json en processed/chunks/.
embedding-index-service	Lee chunks, genera embeddings con all-MiniLM-L6-v2 y almacena vectores y metadatos en ChromaDB.
rag-api-service	Servicio RAG independiente del batch; consulta ChromaDB, arma contexto y llama al LLM para generar una respuesta.

2.3 Dockerización y uso de Amazon ECR

En el proyecto se utilizó Docker para empaquetar cada microservicio con sus dependencias, código fuente y configuración de arranque. El objetivo de la dockerización es que el servicio se ejecute de forma consistente tanto en pruebas locales como en AWS ECS Fargate. Amazon ECR se utiliza como repositorio privado de imágenes Docker, desde donde ECS descarga la imagen correspondiente al momento de ejecutar una tarea.

Paso	Descripción
1. Preparar Dockerfile	Cada servicio cuenta con un Dockerfile que define la imagen base, dependencias, usuario de ejecución, directorio de trabajo y comando de arranque.
2. Construir imagen local	Se ejecuta docker build para construir la imagen del servicio, por ejemplo etl-metadata-service:latest.
3. Crear repositorio ECR	En Amazon ECR se crea un repositorio por servicio o un esquema equivalente para almacenar imágenes Docker.
4. Autenticarse contra ECR	Se usa aws ecr get-login-password para autenticar Docker contra el registro privado de AWS.
5. Etiquetar imagen	La imagen local se etiqueta con la ruta del repositorio ECR, incluyendo account_id, región, repositorio y tag.
6. Publicar imagen	Se ejecuta docker push para subir la imagen a ECR.
7. Usar imagen desde ECS	La task definition de ECS Fargate referencia la imagen publicada en ECR y la descarga al iniciar la tarea.

```
# Ejemplo general por servicio
cd etl-metadata-service
docker build -t etl-metadata-service:latest .

docker tag etl-metadata-service:latest <ACCOUNT_ID>.dkr.ecr.<REGION>.amazonaws.com/etl-metadata-service:latest

aws ecr get-login-password --region <REGION> | docker login --username AWS --password-stdin <ACCOUNT_ID>.dkr.ecr.<REGION>.amazonaws.com

docker push <ACCOUNT_ID>.dkr.ecr.<REGION>.amazonaws.com/etl-metadata-service:latest
```

2.4 Arquitectura actual versus mejora futura

Aspecto	Implementación base	Mejora futura propuesta
Ejecución batch	Tareas ECS Fargate ejecutadas en secuencia manual o programada.	EventBridge programa la ejecución y Step Functions coordina las etapas.
Dependencias entre etapas	Controladas operacionalmente por el administrador o por reglas simples.	Control explícito de estados, errores, reintentos y tiempos máximos.
rag-api-service	Servicio separado que consulta ChromaDB bajo demanda.	Se mantiene fuera de Step Functions porque no es parte del batch.
Observabilidad	Logs en CloudWatch por servicio.	CloudWatch más trazabilidad del flujo de Step Functions.

Step Functions se documenta como mejora futura porque permite una arquitectura más robusta para el procesamiento batch. La solución puede demostrarse ejecutando los tres microservicios batch en orden: etl-metadata-service, chunking-service y embedding-index-service. El rag-api-service se considera independiente porque atiende consultas del usuario una vez que el índice vectorial ya fue construido.

3. Diseño ETL

El diseño ETL organiza el procesamiento documental en etapas separadas, de modo que cada servicio tenga una responsabilidad clara y produzca artefactos intermedios verificables. Esta separación permite auditar el resultado de cada etapa, repetir solo la fase que falló y escalar la solución cuando aumente el número de documentos.

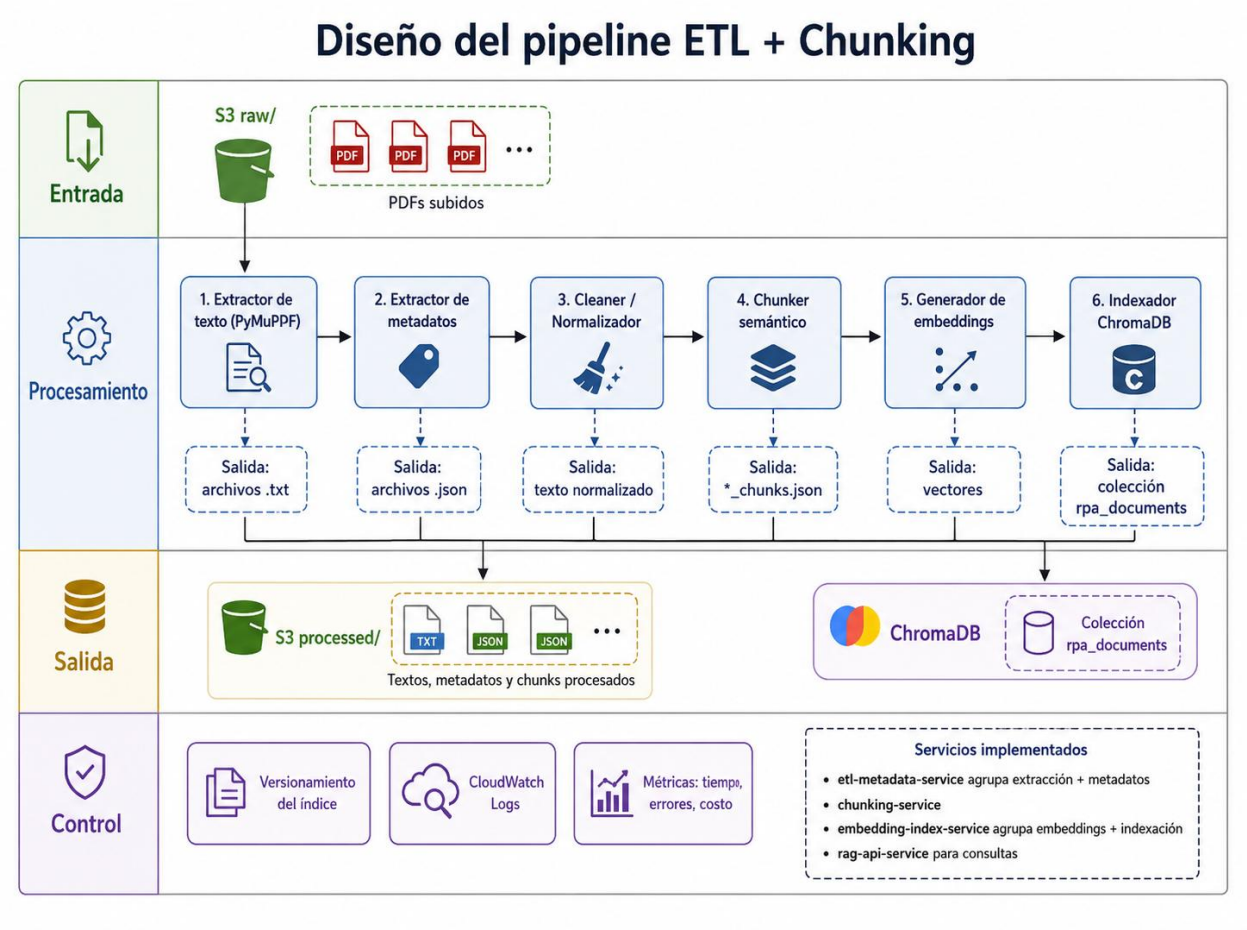


Figura 2. Diseño del pipeline ETL + Chunking.

3.1 Ingesta documental

La ingesta inicia cuando el administrador carga PDFs en el bucket de Amazon S3 bajo el prefijo raw/. Este prefijo representa la zona de entrada del repositorio documental. Los archivos ubicados en raw/ son considerados documentos fuente y no deberían modificarse durante el pipeline; esto permite conservar una copia original y auditable de cada PDF procesado.

3.2 Extracción de texto y metadatos

El servicio etl-metadata-service lee los PDFs desde S3 raw/ y utiliza PyMuPDF para extraer el contenido textual. El texto extraído se almacena en processed/text/, mientras que los metadatos técnicos y documentales se almacenan

en processed/metadata/. Esta etapa permite separar el documento binario original del contenido textual que será usado por las etapas posteriores.

Salida	Descripción	Uso posterior
processed/text/*.txt	Texto plano extraído desde cada PDF.	Entrada principal para el chunking semántico.
processed/metadata/*.json	Datos como nombre de archivo, páginas, fecha de proceso y atributos de origen.	Trazabilidad y enriquecimiento de chunks.

3.3 Limpieza y normalización

La limpieza busca reducir ruido antes de segmentar el texto. En documentos funcionales de RPA pueden existir encabezados repetidos, pies de página, numeración, tablas, saltos de línea inconsistentes y caracteres especiales. Aunque no todo el ruido puede eliminarse sin riesgo de perder información, el pipeline normaliza el texto para mejorar la calidad de los chunks y reducir respuestas irrelevantes en RAG.

- Normalización UTF-8 y eliminación de caracteres de control.
- Reducción de espacios y saltos de línea excesivos.
- Conservación de títulos y secciones cuando aportan contexto semántico.
- Preparación del texto para dividirlo en unidades recuperables.

3.4 Chunking y publicación de artefactos

Una vez normalizado, el texto pasa al chunking-service. Este servicio produce archivos *_chunks.json en processed/chunks/. Cada chunk conserva contenido textual y metadatos mínimos que permiten rastrear su origen. La publicación de chunks en S3 permite inspeccionar la calidad del chunking antes de indexar embeddings.

3.5 Generación de embeddings e indexación

El servicio embedding-index-service toma los chunks generados, produce embeddings mediante all-MiniLM-L6-v2 y los almacena en ChromaDB junto con sus metadatos. Esta etapa transforma el texto en vectores numéricos comparables por similitud, habilitando la recuperación semántica para preguntas del usuario.

4. Patrones LLM

El patrón principal utilizado es Semantic Chunking Pattern, obligatorio para el proyecto. Este patrón no divide el texto únicamente por número de caracteres, sino que busca preservar unidades de significado. En documentos funcionales, una sección como “Gestión de Excepciones” o “Entradas del proceso” debe mantenerse lo más coherente posible para que la respuesta RAG tenga contexto suficiente.

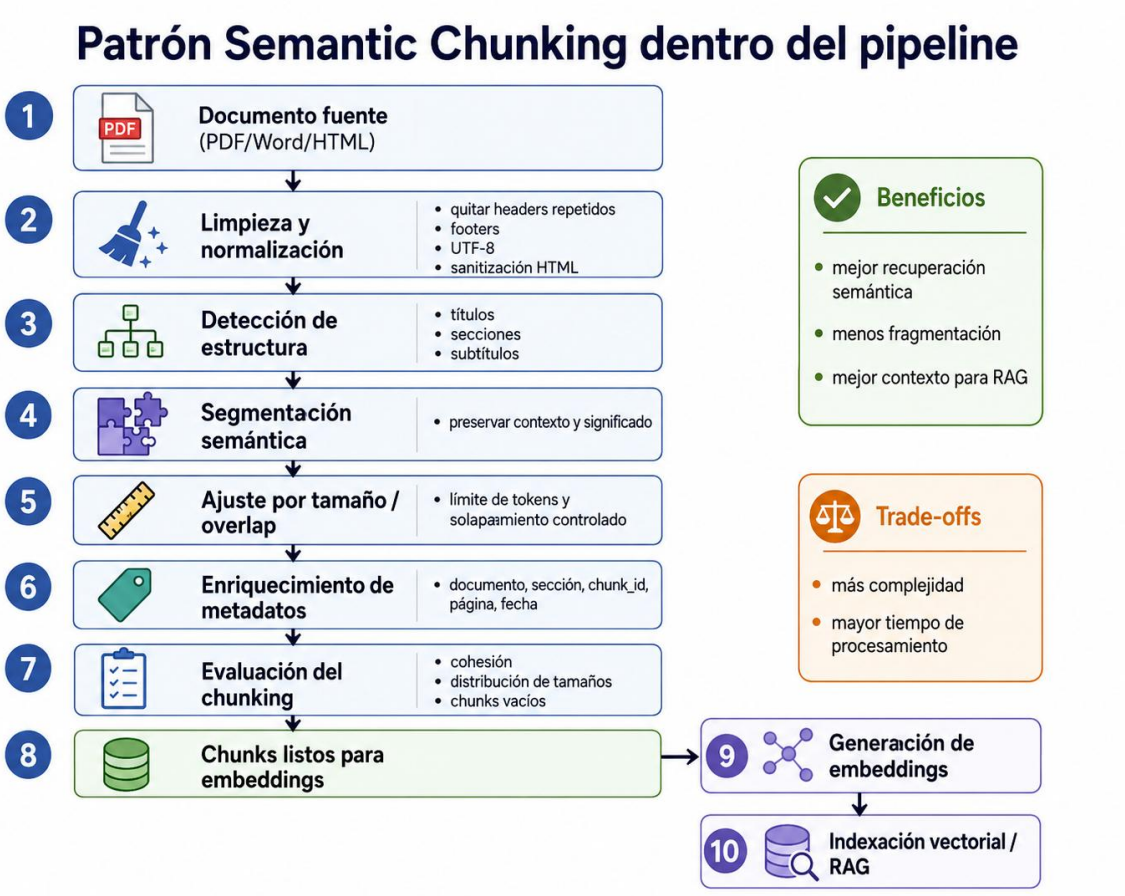


Figura 3. Patrón Semantic Chunking dentro del pipeline.

4.1 Semantic Chunking Pattern

El Semantic Chunking Pattern se aplica detectando estructura documental, secciones, subtítulos y continuidad temática. Cuando el texto se segmenta de forma semántica, los chunks resultantes suelen tener mayor cohesión interna. Esto reduce el riesgo de que una pregunta recupere fragmentos incompletos o demasiado genéricos.

Criterio	Aplicación en el Proyecto
Estructura	Se prioriza el corte por títulos, secciones y bloques funcionales.
Tamaño	Se evita que un chunk sea demasiado pequeño o demasiado grande para la recuperación.
Overlap	Se mantiene solapamiento controlado para no perder contexto entre bloques consecutivos.
Metadatos	Cada chunk conserva datos del documento fuente y su identificación.

4.2 Metadata Enrichment Pattern

Como patrón complementario se considera Metadata Enrichment Pattern. La idea es que cada chunk no solo tenga texto, sino también información adicional que permita filtrar, auditar o explicar su origen. Por ejemplo, document_id, document_name, section_title, chunk_id, indexed_at e index_version.

Este enriquecimiento es especialmente útil cuando se eliminan o reprocesan documentos. Si un PDF cambia, los chunks asociados pueden identificarse por document_id para actualizar el índice sin afectar documentos no relacionados.

4.3 RAG Pattern aplicado a la consulta

El rag-api-service implementa el flujo RAG. Cuando el usuario realiza una pregunta, la API consulta ChromaDB para recuperar los chunks más relevantes, construye un contexto y llama al LLM para redactar una respuesta. El LLM

no responde únicamente desde su conocimiento general, sino usando el contexto recuperado del repositorio documental.

5. Resultados de chunking

Durante las pruebas del proyecto se validó el procesamiento con al menos un PDF funcional real. El pipeline logró extraer texto, generar chunks y preparar la información para indexación vectorial. Uno de los resultados funcionales más relevantes fue la generación de aproximadamente 72 chunks válidos luego de ajustar la estrategia de segmentación.

Este resultado demuestra que el pipeline no se limita a leer un PDF, sino que genera unidades semánticas reutilizables para búsqueda. En un sistema RAG, la calidad de los chunks condiciona directamente la calidad de las respuestas; si los chunks están mal cortados, el sistema puede recuperar información parcial o irrelevante.

Métrica	Resultado / criterio de evaluación
Documentos procesados en prueba	Al menos un PDF funcional real.
Chunks generados	Aproximadamente 72 chunks válidos en la prueba principal.
Formato de salida	*_chunks.json almacenado en processed/chunks/.
Contenido esperado por chunk	Texto, chunk_id, documento fuente, sección o metadatos asociados.
Uso posterior	Entrada para generación de embeddings e indexación en ChromaDB.

5.1 Interpretación de calidad

La calidad del chunking se evalúa observando si cada fragmento conserva una idea funcional completa. Por ejemplo, un chunk relacionado con excepciones debe contener las condiciones, acciones o reglas asociadas, no solo el título. De igual forma, un chunk sobre entradas o salidas debe preservar nombres de archivos, sistemas o campos relevantes.

- Chunks demasiado pequeños pueden perder contexto y generar respuestas incompletas.
- Chunks demasiado grandes pueden introducir ruido y disminuir precisión en la recuperación.
- El overlap ayuda a conservar continuidad entre secciones consecutivas.
- Los metadatos permiten explicar de qué documento proviene cada respuesta.

5.2 Escalabilidad del resultado

Aunque la prueba se realizó con un conjunto reducido de documentos, la arquitectura está preparada para procesar lotes mayores. Al estar desacoplada por S3 y microservicios, se puede ejecutar el pipeline por lotes, reintentar documentos fallidos y ampliar la capacidad de ECS Fargate según la carga. Para un volumen académico mayor, el principal punto de control será el tiempo de procesamiento, el almacenamiento de artefactos intermedios y el crecimiento del índice vectorial.

6. Comparación embeddings

La generación de embeddings convierte cada chunk de texto en un vector numérico. Estos vectores permiten comparar similitud semántica entre una pregunta y los fragmentos del repositorio documental. La selección del modelo de embeddings afecta la calidad de recuperación, el costo, el tiempo de procesamiento, el tamaño del índice y la complejidad operativa.

Modelo	Dimensión	Costo	Ventaja	Limitación
all-MiniLM-L6-v2	384	Sin costo directo de API	Ligero, rápido y adecuado para PoC académica.	Calidad menor que modelos comerciales de mayor dimensión.
AWS Titan Embeddings	Variable	Pago por uso	Integración nativa con AWS	Incrementa costo y

Modelo	Dimensión	Costo	Ventaja	Limitación
			y opción administrada.	requiere configuración adicional.
OpenAI Embeddings	Alta dimensión según modelo	Pago por uso	Muy buena calidad semántica y fácil integración.	Costo variable y dependencia externa.
BGE / E5	Variable	Open-source	Buenas opciones para recuperación semántica avanzada.	Puede requerir mayor capacidad de cómputo.

6.1 Justificación de all-MiniLM-L6-v2

Para este proyecto se eligió all-MiniLM-L6-v2 porque ofrece un equilibrio adecuado entre costo, rendimiento y simplicidad. Al ser un modelo open-source ejecutado dentro del propio servicio, no genera costo directo por cada embedding, lo cual es conveniente para una prueba académica con presupuesto controlado. Además, su dimensión de 384 reduce el tamaño del índice y acelera las búsquedas frente a modelos de mayor dimensión.

La elección también responde al objetivo del proyecto: demostrar el pipeline completo de ingeniería de datos para LLMs. Para una primera versión, era más importante asegurar una cadena funcional de extracción, chunking, embeddings e indexación que maximizar la calidad absoluta del embedding.

6.2 Consideraciones de costo

El uso de un modelo open-source no elimina todos los costos del sistema. El costo se desplaza hacia la infraestructura que ejecuta el modelo: CPU, memoria, almacenamiento y duración de las tareas ECS. En cambio, modelos administrados como OpenAI o Titan simplifican la operación, pero generan un costo variable por volumen de texto procesado.

7. Benchmarking de indexación

La indexación cloud es la etapa que permite consultar los embeddings generados. En este proyecto se seleccionó ChromaDB por su simplicidad, facilidad de integración con Python y bajo esfuerzo de implementación. La decisión es coherente con una PoC académica donde se busca demostrar el flujo completo sin introducir complejidad operativa excesiva.

Opción	Ventajas	Desventajas	Evaluación para el proyecto
ChromaDB	Implementación rápida, integración simple, útil para PoC.	Menor robustez enterprise que servicios administrados.	Seleccionado por simplicidad y velocidad de implementación.
OpenSearch	Escalable, soporta búsqueda híbrida y monitoreo más robusto.	Mayor costo y configuración.	Buena opción futura si se requiere búsqueda híbrida productiva.
Aurora PostgreSQL + pgvector	Combina SQL y vectores, útil para trazabilidad transaccional.	Requiere diseño de base de datos y administración adicional.	Alternativa futura si se necesita gobierno de datos más formal.
Azure AI Search	Servicio administrado con capacidades de búsqueda avanzadas.	No está alineado al stack AWS elegido.	Descartado por consistencia con AWS.

7.1 Criterios de evaluación

- Latencia de búsqueda semántica para recuperar los top-k chunks más relevantes.
- Facilidad de integración con el servicio RAG.
- Costo de operación y almacenamiento del índice.

- Capacidad de filtrar por metadatos como document_id, versión o estado.
- Complejidad de administración y despliegue dentro de AWS.

ChromaDB cumple adecuadamente con los criterios mínimos para una demostración funcional. Permite almacenar embeddings, asociarlos a metadatos y consultarlos por similitud. Para una solución productiva, el benchmark debería ampliarse midiendo tiempos de inserción, tiempos de consulta, crecimiento del índice, recuperación híbrida y comportamiento ante eliminación o reprocesamiento de documentos.

7.2 Relación con el rag-api-service

El rag-api-service depende directamente de ChromaDB para recuperar contexto. Cuando un usuario pregunta, la API consulta el índice, obtiene los chunks más cercanos semánticamente y los utiliza como contexto para el LLM. Por esta razón, el índice vectorial no debe verse como un almacenamiento accesorio, sino como el componente central que conecta el pipeline batch con la experiencia de consulta RAG.

8. Versionamiento

El versionamiento es necesario porque el repositorio documental cambia con el tiempo. Pueden agregarse nuevos PDFs, actualizarse documentos existentes o eliminarse documentos obsoletos. Si el índice vectorial no registra esos cambios, el RAG podría responder con información desactualizada o inconsistente.

La estrategia propuesta consiste en registrar una versión lógica del índice cada vez que se ejecuta un lote relevante de procesamiento. Esta versión debe almacenar información mínima como fecha de indexación, cantidad de documentos, cantidad de chunks, cantidad de embeddings, hash del dataset y estado de la ejecución.

Evento	Comportamiento esperado
Alta de documento	El PDF se incorpora en raw/, se procesa, se generan chunks y se agregan embeddings al índice.
Reprocesamiento	Se identifican los chunks anteriores del documento y se reemplazan por una nueva versión.
Eliminación	Se elimina o marca inactivo el documento y se eliminan o excluyen sus embeddings asociados.
Error de procesamiento	El documento se registra como fallido y no debe participar en respuestas RAG hasta corregirse.

8.1 Identificadores recomendados

```
{
  "document_id": "formato_4_2_generacion_cronogramas",
  "document_name": "Formato 4.2- Generación de Cronogramas PN.pdf",
  "chunk_id": "formato_4_2_chunk_001",
  "index_version": "v1",
  "status": "ACTIVE",
  "indexed_at": "2026-06-09"
}
```

Con estos identificadores, el sistema puede rastrear qué chunks pertenecen a cada documento y qué versión del índice los contiene. Esto facilita auditoría y mantenimiento, especialmente cuando se eliminan documentos o se corrigen PDFs mal procesados.

8.2 Eliminación de documentos en RAG

Si un PDF se elimina de S3 raw/ pero sus chunks y embeddings permanecen en ChromaDB, la aplicación podría seguir respondiendo con información antigua. Por eso, la eliminación debe propagarse a los artefactos procesados y al índice vectorial. La eliminación puede ser física, borrando los registros del índice, o lógica, marcándolos como inactivos y filtrándolos durante la búsqueda.

1. Eliminar o marcar el PDF como inactivo en S3 raw/.
2. Eliminar artefactos asociados en processed/text/, processed/metadata/ y processed/chunks/.
3. Eliminar o desactivar embeddings asociados al document_id en ChromaDB.
4. Registrar una nueva versión del índice o evento de cambio.
5. Validar que rag-api-service ya no recupere chunks del documento eliminado.

8.3 Step Functions como mejora futura del versionamiento

Step Functions puede mejorar el versionamiento porque permite modelar estados de ejecución. Por ejemplo, una ejecución puede pasar por estados como ETL_COMPLETED, CHUNKING_COMPLETED, INDEXING_COMPLETED o FAILED. Esto ayuda a registrar con precisión qué documentos fueron procesados y en qué etapa fallaron. En esta entrega se considera una mejora futura, no un requisito obligatorio de la implementación base.

9. Lecciones aprendidas

El desarrollo del proyecto permitió validar que un pipeline RAG no depende únicamente del modelo LLM. La calidad de la respuesta final está condicionada por todas las etapas anteriores: extracción, limpieza, chunking, embeddings, indexación y recuperación. Un error en cualquiera de esas fases puede generar respuestas incompletas o irrelevantes.

Lección	Explicación
El chunking es crítico	La segmentación define qué contexto será recuperado por el RAG. Chunks mal diseñados reducen la calidad de respuesta.
La arquitectura debe separar batch y consulta	El procesamiento documental es batch; la API RAG es bajo demanda y debe mantenerse independiente.
ECR no es parte del flujo de datos	ECR almacena imágenes Docker; los microservicios ejecutados en ECS son los que leen o escriben datos.
La observabilidad debe diseñarse desde el inicio	Logs y métricas permiten explicar fallas y evidenciar ejecución ante el evaluador.
Step Functions aporta robustez futura	Permite automatizar dependencias, reintentos y trazabilidad del pipeline batch.

9.1 Conclusión técnica

La solución propuesta cumple el objetivo de diseñar e implementar un pipeline de ingeniería de datos para LLMs. A partir de PDFs funcionales, el sistema extrae texto, genera metadatos, aplica chunking semántico, produce embeddings, indexa en ChromaDB y habilita una consulta RAG mediante rag-api-service. La arquitectura prioriza simplicidad, bajo costo y capacidad de demostración, pero deja claras las rutas de evolución hacia una solución más productiva.

9.2 Trabajo futuro

- Implementar Step Functions para orquestar formalmente ETL, chunking e indexación.
- Agregar manejo automático de documentos fallidos y reintentos por etapa.
- Implementar versionamiento persistente del índice en una tabla de control.
- Evaluar modelos de embeddings de mayor calidad para mejorar recall semántico.
- Migrar ChromaDB a una alternativa administrada si se requiere operación productiva.
- Agregar evaluación cuantitativa con métricas de precisión, recall o RAGAS.

10. Actualización posterior: persistencia ChromaDB vía S3

Durante las pruebas finales en ECS/ECR se detectó un punto crítico: si ChromaDB se genera dentro de una tarea batch de Fargate, el almacenamiento local del contenedor se pierde al finalizar la tarea. Por eso, rag-api-service no debe

depender de una carpeta local generada en otra tarea ni de una copia de ChromaDB incluida dentro de la imagen Docker.

La implementación actual usa Amazon S3 como mecanismo de intercambio persistente entre el batch de indexación y la API RAG. embedding-index-service genera la base vectorial en temp/chroma_db, sube la carpeta completa al prefijo processed/chroma_db/ y finaliza. Luego, rag-api-service, al iniciar, descarga processed/chroma_db/ desde S3 hacia su ruta local CHROMA_DB_PATH antes de crear el cliente PersistentClient de ChromaDB.

Componente	Responsabilidad actualizada
embedding-index-service	Genera embeddings, actualiza ChromaDB local en temp/chroma_db y sube la carpeta completa a S3 processed/chroma_db/.
Amazon S3 processed/chroma_db/	Actúa como repositorio persistente del índice ChromaDB entre ejecuciones batch y la API.
rag-api-service	Restaura ChromaDB desde S3 al iniciar, luego abre la colección rpa_documents y atiende /ask.
Docker / ECR	Publica código y dependencias, pero no debe incluir temp/chroma_db dentro de la imagen.

Diagrama Mermaid actualizado del flujo de persistencia:

```
flowchart LR
    A[PDFs en S3 raw/] --> B[etl-metadata-service]
    B --> C[S3 processed/text y processed/metadata]
    C --> D[chunking-service]
    D --> E[S3 processed/chunks]
    E --> F[embedding-index-service]
    F --> G[temp/chroma_db local]
    G --> H[S3 processed/chroma_db]
    H --> I[rag-api-service al iniciar]
    I --> J[ChromaDB local restaurado]
    J --> K[Endpoint POST /ask]
    K --> L[OpenAI gpt-4.1-nano]
    K --> M[Respuesta con fuentes recuperadas]

    N[Docker build] --> O[Amazon ECR]
    O --> P[ECS Fargate task definitions]
    P --> B
    P --> D
    P --> F
    P --> I
```

Esta decisión mantiene la demostración académica simple y funcional en Fargate sin introducir todavía EFS, Aurora pgvector u OpenSearch. Para producción, EFS o un vector store administrado podrían reemplazar este intercambio por S3, pero para la entrega actual el flujo S3 -> ChromaDB local restaurado es suficiente y verificable.

10.1 Implicancias de Docker y .dockerignore

- La carpeta temp/chroma_db debe permanecer ignorada en .dockerignore para rag-api-service, porque el índice se restaura desde S3 al iniciar.
- El Dockerfile de rag-api-service no debe contener COPY temp/chroma_db /app/temp/chroma_db ni instrucciones equivalentes que metan la base vectorial dentro de la imagen.
- Cada cambio de código requiere rebuild, tag, push a ECR y actualización de la task definition o del ECS Service para que Fargate ejecute la nueva imagen.
- Si se usa el tag latest, ECS puede requerir nueva revisión de task definition o force new deployment para descargar la imagen actualizada.

10.2 Variables de entorno actualizadas

Servicio	Variables clave
embedding-index-service	AWS_REGION=us-east-1; BUCKET_NAME=<bucket>; CHROMA_DB_PATH=temp/chroma_db; CHROMA_COLLECTION=rpa_documents; CHROMA_S3_PREFIX=processed/chroma_db
rag-api-service	AWS_REGION=us-east-1; BUCKET_NAME=<bucket>; CHROMA_DB_PATH=temp/chroma_db o /app/temp/chroma_db; CHROMA_COLLECTION=rpa_documents; CHROMA_S3_PREFIX=processed/chroma_db; OPENAI_MODEL=gpt-4.1-nano

10.3 Actualización de ECS Service después de publicar una nueva imagen

Cuando se modifica rag-api-service o embedding-index-service, no basta con hacer docker push. ECS debe ejecutar una nueva revisión de task definition o forzar un nuevo despliegue del servicio para tomar la imagen más reciente.

```
# Rebuild y publicación de rag-api-service
cd rag-api-service
docker build -t rag-api-service:latest .
docker tag rag-api-service:latest 312605973951.dkr.ecr.us-east-1.amazonaws.com/rag-api-service:latest
docker push 312605973951.dkr.ecr.us-east-1.amazonaws.com/rag-api-service:latest

# Luego actualizar la task definition a la nueva revisión desde consola AWS
# o forzar nuevo despliegue si el service ya apunta a la revisión correcta:
aws ecs update-service   --cluster rpa-llm-cluster   --service rag-api-service   --force-new-deployment   --region us-east-1
```

Si la nueva configuración de variables de entorno o el comando del contenedor cambió, se debe registrar una nueva revisión de task definition y actualizar el servicio para usar esa revisión. Por ejemplo, si la revisión vigente corregida es :3, el servicio debe apuntar a rag-api-service-task:3.